

Heap 与优先队列

不维护完全有序，怎样持续取得当前极值

408 · 数据结构 Topic DS-05

母问题：为什么只维护局部有序就够了？

堆的生成链是

Priority Workload → Complete-Tree Layout → Root Extremum → Local Repair → Heap Invariant → Cost.

最小堆只保证每个父节点不大于孩子，因此根是全局最小值，但任意两个非祖先节点没有排序承诺。优先队列需要的正是“看极值、删极值、加入新候选”，而不是随时输出完整有序序列。

本册定位与 Owner 边界

- **Owns**：完全二叉树数组布局、父子下标、上滤、下滤、建堆、插入、删除极值与优先队列接口。
- **Uses**：Atlas Foundation 的操作成本向量；DS04 Huffman 和 DS08 图算法只调用优先队列接口。
- **Stops**：一般树遍历归 DS04；heap sort 的整体过程由 DS11 组织；Prim/Dijkstra 的目标不变量归 DS08。

目录

1	表示与核心不变量	2
2	四种状态转移	2
3	优先队列与外部目标	2
4	代码与测试证据	2
5	复杂度、边界与概念分离	4
6	做题控制协议	4

1 表示与核心不变量

完全二叉树按层序放入数组。零基下标下，节点 i 的父节点为 $(i-1)/2$ ，孩子为 $2i+1, 2i+2$ ；越界孩子表示不存在。最小堆不变量为

$$\forall i > 0, \quad A[\text{parent}(i)] \leq A[i].$$

它只承诺根极值；把堆误读成有序数组，是许多“第二小元素/任意两元素比较”错误的来源。

局部有序不等于全局有序

数组 $[2, 5, 3, 9, 7, 8]$ 是最小堆，但 5 与 3 的相对位置没有排序意义。取最小值只看根，删除根后把末元素搬到根并下滤，结构仍保持完全树形。

2 四种状态转移

插入先把新元素放在数组末尾，再与父节点比较并上滤；每次交换都使元素向根移动一层。删除极值先保存根，把末元素搬到根并缩短长度，再与较小孩子交换并下滤。两种修复都只沿一条根叶路径进行，路径长度为 $h = \Theta(\log n)$ 。

建堆不应逐个调用插入来默认获得最优界。对最后一个非叶节点向前执行下滤，每个子树的高度不同，总成本为 $O(n)$ ；逐个插入则为 $O(n \log n)$ 。这两个结论的前提不同，不能只背“堆是 $\log n$ ”。

3 优先队列与外部目标

优先队列抽象为 `push`、`top`、`pop`、`empty`。`Huffman` 需要反复取两个最小权值，`Dijkstra/Prim` 需要取当前最小候选；它们不应复制下滤代码。若优先级相等，是否稳定需要额外的序号字段，堆本身不自动提供稳定性。

4 代码与测试证据

完整实现位于 `code/ds05_heap.hpp`，测试位于 `code/ds05_heap_test.cpp`。代码把“末尾插入—上滤”和“根替换—下滤”分成两个可检查的状态转移。

Listing 1: 堆布局、上滤、下滤与极值接口

```

14     build();
15 }
16
17 bool empty() const { return data_.empty(); }
18 std::size_t size() const { return data_.size(); }
19 int top() const {
20     if (empty()) throw std::out_of_range("top on empty heap");
21     return data_.front();
22 }
23 void push(int value) {
24     data_.push_back(value);
25     sift_up(data_.size() - 1);
26 }
27 int pop() {

```

```

28     const int result = top();
29     data_.front() = data_.back();
30     data_.pop_back();
31     if (!empty()) sift_down(0);
32     return result;
33 }
34 bool invariant() const {
35     for (std::size_t i = 1; i < data_.size(); ++i) {
36         if (data_[parent(i)] > data_[i]) return false;
37     }
38     return true;
39 }
40
41 private:
42     std::vector<int> data_;
43     static std::size_t parent(std::size_t i) { return (i - 1) / 2; }
44     static std::size_t left(std::size_t i) { return 2 * i + 1; }
45
46     void sift_up(std::size_t i) {
47         while (i > 0 && data_[parent(i)] > data_[i]) {
48             std::swap(data_[parent(i)], data_[i]);
49             i = parent(i);
50         }
51     }
52     void sift_down(std::size_t i) {
53         while (left(i) < data_.size()) {
54             std::size_t child = left(i);
55             if (child + 1 < data_.size() && data_[child + 1] < data_[child]) ++
child;
56             if (data_[i] <= data_[child]) break;
57             std::swap(data_[i], data_[child]);
58             i = child;
59         }
60     }
61     void build() {
62         if (data_.empty()) return;
63         for (std::size_t i = data_.size() / 2; i-- > 0;) sift_down(i);
64     }
65 };
66
67 } // namespace ipara::ds05
68
69 #endif

```

Listing 2: 空堆、单元素与重复优先级测试

```

12
13 MinHeap heap({7, 2, 9, 1, 2});

```

```
14  assert(heap.invariant());
15  heap.push(0);
16  assert(heap.top() == 0 && heap.invariant());
17  assert(heap.pop() == 0);
18  assert(heap.pop() == 1);
19  heap.push(3);
20  assert(heap.invariant());
21  assert(heap.pop() == 2);
22  assert(heap.pop() == 2);
23  assert(heap.pop() == 3);
24  assert(heap.pop() == 7);
25  assert(heap.pop() == 9);
26  assert(heap.empty());
27 }
```

```
clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/ds05_heap_test.
    cpp -o /tmp/ds05_test
/tmp/ds05_test
```

5 复杂度、边界与概念分离

操作	时间	额外空间	不变量动作	边界
top	$O(1)$	$O(1)$	只读根	空堆不可取值
push/pop	$O(\log n)$	$O(1)$	上滤/下滤	单元素删除
建堆	$O(n)$	$O(1)$	自底向上	空输入

概念 A	≠ 概念 B	判据	错因
堆序	≠ 完全有序	只比较父子	错误推出任意排名
建堆	≠ 逐个插入	自底向上 vs 重复上滤	误报复杂度
优先队列	≠ 堆实现	接口 vs 具体表示	把调用方绑死

6 做题控制协议

先问目标是否只需当前极值；若是，选择优先队列。再画完全树索引，插入看“末尾后上滤”，删除看“末尾补根后下滤”。每次交换后检查父子关系和数组边界；若题目需要稳定次序、任意第 k 小或完整排序，停止把堆当万能索引，转交 DS09/DS11。

压缩复原

忘记代码时复原五问：根承诺什么？新元素从哪里进入？删除后谁补根？冲突孩子选哪一个？修复结束条件是什么？答案分别恢复根极值、数组末尾、末元素、较小孩子和父子关系成立。